

Que :- To study printer deadlock scenarios, analyze resource contention, and propose synchronization strategies ensuring smooth, efficient printing operations.

Code:

```

<!-- You, 9 minutes ago | 1 author (You) -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Deadlock in Operating Systems</title>
  <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;800&display=swap" rel="stylesheet">
  <style>
    :root {
      --primary: #6366f1;
      --secondary: #a855f7;
      --dark: #0f172a;
      --card-bg: rgba(30, 41, 59, 0.7);
      --text: #f8fafc;
    }

    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: 'Outfit', sans-serif;
    }

    body {
      background: linear-gradient(135deg, #0f172a 0%, #1e1b4b 100%);
      color: var(--text);
      min-height: 100vh;
      overflow-x: hidden;
    }

    /* Ambient Glow Background */
    .ambient-glow {
      position: fixed;
      top: 50%;
      left: 50%;
      width: 80vw;
      height: 80vh;
      transform: translate(-50%, -50%);
      background: radial-gradient(circle, #6366f1 0%, #a855f7 70%);
      z-index: -1;
      pointer-events: none;
    }

    /* Navigation Bar */
    nav {
      background: #0f172a;
      padding: 1.2rem 0;
      position: fixed;
      width: 100%;
      top: 0;
      z-index: 1000;
      backdrop-filter: blur(12px);
      border-bottom: 1px solid #6366f1;
    }

    nav ul {
      list-style: none;
      display: flex;
      justify-content: center;
      align-items: center;
      gap: 2rem;
    }

    nav ul li a {
      color: #f8fafc;
      text-decoration: none;
      font-weight: 600;
      font-size: 1.1rem;
    }

    .car-elem {
      font-size: 1.5rem;
      z-index: 20;
      transition: left 0.5s linear, top 0.5s linear;
      box-shadow: 0 10px 20px #0f172a;
      opacity: 0;
    }

    .car-v { width: 40px; height: 70px; }
    .car-h { width: 70px; height: 40px; }

    .car-elem.visible {
      opacity: 1;
    }

    /* 4 Conditions Cards */
    .conditions {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 1.5rem;
    }

    .condition-card {
      background: #0f172a;
      border: 1px solid #6366f1;
      padding: 2rem;
      border-radius: 20px;
      transition: all 0.3s ease;
    }

    .condition-card:hover {
      transform: translateY(-5px);
      background: #0f172a;
      border-color: #6366f1;
    }

    </style>
  </head>
  <body>
    <div class="ambient-glow"></div>
    <div class="deadlock-overlay" id="deadlockOverlay"></div>

    <div class="alert-box" id="alertBox">
      <h2>DEADLOCK DETECTED!</h2>
      <p style="color: #c0392b; font-size: 1.1rem; margin-bottom: 2rem; max-width: 400px; line-height: 1.5; margin-left: auto; margin-right: auto;">
        All 4 conditions are met! Cars wait in a circular chain. No one can proceed because the intersection is permanently blocked.
      </p>
      <button class="reset-btn" style="margin: 0 auto;" onclick="resetSimulation()">
        Resolve & Reset
      </button>
    </div>

    <!-- Navigation Bar -->
    <nav>
      <ul>
        <li><a class="nav-link active" onclick="showPage('start', this)">Home</a></li>
        <li><a class="nav-link" onclick="showPage('deadlock', this)">Simulation</a></li>
        <li><a class="nav-link" onclick="showPage('deadlock', this)">4 Conditions</a></li>
      </ul>
    </nav>

    <div class="container">
      <!-- Page 1: Start Page -->
      <div id="start" class="start-page">
        <h1 class="start-title">OS Deadlock</h1>
        <p class="subtitle">
          A visual journey into how processes halt and the 4 Coffman conditions that cause Operating System deadlocks.
        </p>
        <button class="primary-btn" onclick="showPage('deadlock', document.querySelector('.nav-link')[1])">
          Initialize Simulation
        </button>
      </div>
    </div>
  </body>
</html>

```

```

// Base positions for each direction [top, left]
const CAR_POS = {
  north: { start: [20, 200], wait: [110, 200], end: [440, 200] },
  south: { start: [410, 260], wait: [320, 260], end: [20, 260] },
  west: { start: [260, 20], wait: [260, 110], end: [260, 440] },
  east: { start: [200, 410], wait: [200, 320], end: [200, 20] }
};

function showPage(pageId, navElement) {
  document.querySelectorAll('.start-page, .deadlock-page').forEach(page => {
    page.style.display = 'none';
  });
  document.getElementById(pageId).style.display = 'block';

  document.querySelectorAll('.nav-link').forEach(link => link.classList.remove('active'));
  if(navElement) navElement.classList.add('active');

  if (pageId === 'deadlock') {
    if(!intervalId) resetSimulation();
  } else {
    clearInterval(intervalId);
    intervalId = null;
  }
}

// Set light explicitly
function setLight(dir, color) {
  ['r', 'y', 'g'].forEach(c => {
    const el = document.getElementById(`tl-${dir}-${c}`);
    el.style.background = `#${333}`;
    el.style.boxShadow = 'none';
  });
}

const target = document.getElementById(`tl-${dir}-${color[0]}`);

```

```

function triggerDeadlock() {
  clearInterval(intervalId);

  document.getElementById('btnDeadlock').disabled = true;
  document.getElementById('btnDeadlock').style.opacity = '0.5';

  // Set all lights to red
  ['n', 's', 'w', 'e'].forEach(dir => setLight(dir, 'red'));

  // All cars move to wait point
  ['north', 'south', 'west', 'east'].forEach(dir => {
    const el = document.getElementById(`car-${dir}`);
    el.style.transition = 'left 1s cubic-bezier(0.2, 0.8, 0.2, 1), top 1s cubic-bezier(0.2, 0.8, 0.2, 1)';
    // Position so they are blocking each other directly in intersection:
    let x = CAR_POS[dir].wait[1]; let y = CAR_POS[dir].wait[0];
    if(dir==='north') y=200;
    if(dir==='south') y=260;
    if(dir==='west') x=200;
    if(dir==='east') x=260;

    el.style.top = y + 'px';
    el.style.left = x + 'px';
  });

  setTimeout(() => {
    document.getElementById('deadlockOverlay').style.display = 'block';
    document.getElementById('alertBox').classList.add('show');
  }, 1200);
}
</script>
</body>
</html>

```

Output:

